

Trabajo final Inteligencia Artificial 1

Clasificación de piezas de ferretería mediante visión artificial

Año 2021

Alumno

Miled Felicito Gomez

Legajo: 11901

Profesora Titular

Dra. Ing. Selva S. Rivera

Fecha

Julio del 2024

Resumen

Este proyecto de visión artificial consiste en la clasificación de piezas metálicas de ferretería como tornillos, clavos, tuercas y arandelas, a través de una fotografía.

El algoritmo cuenta con distintas etapas, las cuales se pueden resumir en: lectura de imagen, transformación para lograr la normalización y binarización, extracción de características como momentos de Hu, área, perímetro y circularidad, y clasificación. Se emplean algoritmos KNN y K-Means de autoría propia que se entrenan mediante una base de datos de los distintos objetos mencionados, y para el caso de tornillos y clavos se usa un algoritmo para proporcionar su longitud aproximada. Todo fue realizado en el lenguaje de programación Python.

Introducción

Un concepto fundamental a tener en cuenta para el desarrollo y planificación de este proyecto es la visión artificial:

“La visión artificial es un campo interdisciplinario de la informática y la inteligencia artificial que busca dotar a las máquinas con la capacidad de interpretar y comprender el mundo visual de manera similar a como lo hacen los seres humanos. Utilizando algoritmos y técnicas sofisticadas, la visión artificial permite a los sistemas computacionales analizar y procesar imágenes o videos para extraer información significativa. Este campo tiene aplicaciones en diversas industrias, como la medicina, la manufactura, la seguridad, el transporte y el entretenimiento, y ha experimentado un rápido avance en las últimas décadas gracias a los desarrollos en algoritmos de aprendizaje profundo y el incremento en la capacidad de procesamiento de hardware.”

Así es como, por medio del conocimiento de esta disciplina científica, se tomarán imágenes del mundo real de los diferentes elementos a clasificar, para luego de un particular procesado obtener características representativas de cada una. Los cuáles serán sometidos a métodos de comparación numéricos, obteniendo finalmente una probable identificación de la imagen. Muchas veces no se conseguirá el mejor de todos los resultados, pero nos conformamos con lo mejor que se pueda obtener dentro de ciertos parámetros.

Especificación del agente

Tipo de agente

Es un agente que aprende. El aprendizaje permite que el agente opere en medios inicialmente desconocidos y que sea más competente que si sólo utilizase un conocimiento inicial. En este caso aprende debido a los métodos que utiliza, KNN y K-Means. La base de datos le proporciona un “modelo” de su entorno, también los algoritmos en base a la “utilidad”.

También el agente es racional esto implica: “ En cada posible secuencia de percepciones, un agente racional deberá emprender aquella acción que supuestamente maximice su medida de rendimiento, basándose en las evidencias aportadas por la secuencia de percepciones y en el conocimiento que el agente mantiene almacenado”. Toma decisiones para maximizar su rendimiento, basándose en la información que recibe de su entorno y en su conocimiento previo.

También es un agente no omnisciente debido a que no conoce el resultado de su acción y no actúa de acuerdo a él. La idea es que a partir de los algoritmos KNN y K-Means se entrene, aprenda a partir de sus percepciones y le permita ser autónomo gracias a la experiencia adquirida. Toma decisiones basadas en la información disponible en lugar de conocer los resultados futuros.

Para este proyecto el agente es una computadora donde están alojados los algoritmos KNN y K-Means. Los datos de entrada y salida son adquiridos por cámaras fotográficas, monitor, etc.

REAS

El acrónimo REAS proviene de Rendimiento, Entorno, Actuadores y Sensores, todo ello forma parte de lo que se llama Entorno de trabajo.

Rendimiento	-Confiablez de las predicciones con un margen de error pequeño. -Cantidad de predicciones correctas de las imágenes que se están evaluando. -Categorización de imagen correcta.
Entorno	-Es el contexto plasmado en la imagen de entrada. Incluye factores como la iluminación, calidad de la imagen, fondo, etc.
Actuadores	-Es el medio de notificación del usuario donde se muestra la predicción del algoritmo. En este caso es el monitor de la computadora. -Visualizar la categorización de una imagen.

Sensores	-Cámara fotográfica. -Módulos cv2, numpy, matplotlib, etc.
----------	---

Propiedades del entorno de trabajo

Totalmente observable	Los sensores previamente mencionados le proporcionan acceso al estado completo del medio en cada momento.
Determinista	El estado del medio está totalmente determinado por el estado actual y la acción ejecutada del medio. La imagen de entrada una vez tomada es inalterable en principio. Lo que sucede a continuación en el entorno está completamente determinado por lo que hace el agente y el estado actual.
Episódico	La elección de la acción en cada episodio depende del episodio en sí mismo. Uno puede tomar las fotografías en cualquier orden de tornillos, clavos, arandelas o tuercas y el rendimiento del agente no se verá afectado. El agente toma decisiones basadas en la situación actual sin importar lo que pasó antes.
Estático	El entorno no cambia cuando el agente está deliberando. Una vez tomada la foto no cambia. En la fotografía, ese dato de entrada se mantiene inalterable.
Discreto	Las imágenes captadas por las cámaras son discretas, en sentido estricto, pero se tratan típicamente como representaciones continuas de localizaciones e intensidades variables.
Individual	El único ente racional es el agente previamente descrito.

Diseño del agente

A continuación se hará mención y explicación de las distintas etapas del agente.

1) Base de datos

Previamente se sacan fotos de los distintos elementos bajo condiciones controladas y lo más homogéneas posible, se procesan mediante el programa "Procesamiento_img", que será detallado posteriormente, y luego se guardan en carpetas según el tipo de pieza. En este caso se sacaron 5 fotos de cada objeto que luego se utilizaron para entrenar los algoritmos. En total fueron 20 imágenes de entrenamiento.

2) Cargar imagen

Se realiza la toma de la fotografía del objeto puesto dentro de una caja de zapatillas con una hoja negra de fondo. Luego se descarga la imagen del celular y se pasa manualmente al programa.

3) Procesamiento

En esta parte lo que se hace es tratar a la imagen con diferentes técnicas para que quede lo más limpia posible para su reconocimiento.

Se hace lo siguiente:

- Cargar la imagen: se carga la imagen de entrada utilizando la función `cv2.imread()`. Esta imagen se almacenará en la variable "image".
- Remover fondo: separa el fondo del objeto y lo elimina.
- Convertir a escala de grises: la imagen se convierte de una imagen a color a una imagen en escala de grises utilizando `cv2.cvtColor()`.
- Desenfocar: se aplica un filtro Gaussiano para reducir el ruido.
- Binarización de la imagen: la imagen en escala de grises se binariza utilizando el umbral de OTSU. Esto se logra mediante la función `cv2.threshold()` con los parámetros apropiados. Además se usa `cv2.adaptiveThreshold`, un umbral adaptativo, para mejorar la binarización

4) Características calculadas

Un momento es una cantidad, una cifra que captura información básica de la forma de un objeto en una imagen, como su área, las coordenadas de su centroide, su orientación, entre otros, un sinnúmero de propiedades interesantes. Son siete los valores (momentos), los cuales no se ven afectados por cambios en la rotación, traslación, escala y reflexión de una imagen.

En este caso se terminan usando solo tres valores de los mencionados debido a su relevancia superior. Además se extraen otras características como área, perímetro, y claridad que ayudan a separar mejor los tipos de elementos, dando un total de seis valores. Se crea un clase "Elemento" donde se guarda la información proporcionada para cada grupo de piezas.

5) Clasificación

Esta tarea es realizada por los dos algoritmos anteriormente mencionados (Knn y K-Means). Se proporciona una base de datos con la cual se entrenan y luego se compara para obtener un resultado.

KNN

K-Nearest Neighbors (KNN), también llamado el vecino más cercano, puede ser utilizado para clasificación y problemas de regresión predictiva, en la industria es más utilizado para problemas de clasificación, donde la mayoría de los conjuntos de datos del mundo real no siguen supuestos teóricos matemáticos.

Implementación: Facilidad para interpretar las salidas (predicciones), bajo tiempo de cálculo y el poder de predicción.

Pasos del algoritmo KNN

- 1) Comparar las características de la imagen a evaluar con las de todas las muestras del conjunto de entrenamiento y se calcula la distancia euclidiana de la resta de las mismas.
- 2) Se van comparando las distancias y se asigna la etiqueta del elemento que tiene la menor de las distancias.

El número de vecinos (K) en KNN, es un hiperparámetro que debe elegir en el momento de la construcción del modelo. Se puede pensar en K como una variable de control para el modelo de predicción.

En el caso de una pequeña cantidad de vecinos, el ruido tendrá una mayor influencia en el resultado, y una gran cantidad de vecinos lo hará computacionalmente costoso. En este caso se adoptó $k = 5$.

KMeans

Es un algoritmo de agrupación que divide las observaciones en K agrupaciones. La agrupación es un tipo de aprendizaje automático no supervisado que tiene como objetivo encontrar subgrupos homogéneos de manera que los objetos en el mismo grupo (agrupaciones) sean más similares entre sí que los demás.

Pasos del algoritmo K-Means

- 1) Se elige la localización de los centroides seleccionando aleatoriamente un elemento de los datos y tomando sus características como la media inicial para ese tipo de pieza.
- 2) Se calcula la distancia del elemento a cada una de las medias o centroides, eligiendo la mínima de estas.
- 3) Se asigna el elemento al cluster cuya distancia es la mínima, es decir, el cluster cuyo centroide está más cercano.
- 4) Se actualizan los centroides de cada cluster luego de asignar todos los elementos. Esto se repite hasta la convergencia, que ocurre cuando las medias anteriores y actualizadas son iguales.

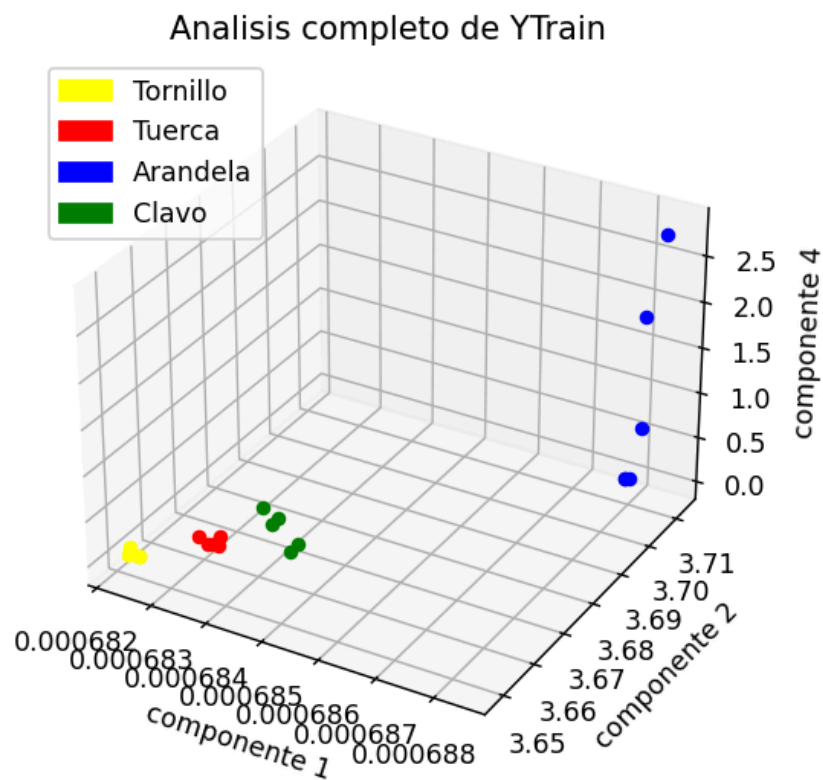
Se recomienda normalizar los datos, ya que ayuda al clustering porque los grupos se forman a partir de la distancia euclidiana cuadrada.

6) Medición de tornillos y clavos

Se requiere hacer la medición de la longitud de las piezas que fueran tornillos o clavos. Para esto se utiliza la imagen ya procesada y se pasa por una función llamada "calcular_longitud" donde se le hace un procesamiento adicional que consiste en:

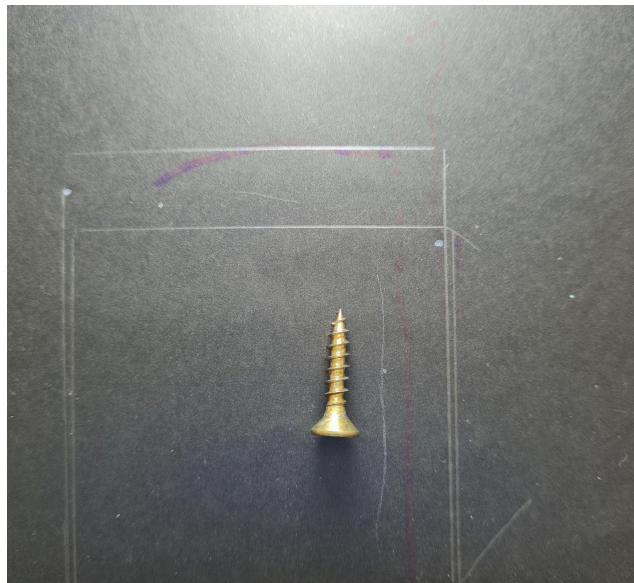
1. Detectar los bordes de la misma
2. Encontrar los contornos
3. Tomar el contorno con el área más grande y calcularla
4. Calcular dimensiones del rectángulo delimitador
5. Tomar la longitud como el lado más largo de ese rectángulo
6. Hacer conversión de pixeles a milímetros

Ejemplo de aplicación y resultados



1) Ejemplo Tornillo

Se analizó el tornillo de la siguiente foto:



El programa arroja los siguientes resultados:

```
PS D:\DB\IA_Proyecto> & C:/Users/miled/AppData/Local/Programs/Python/Python311/python.exe d:/DB/IA_Proyecto/clasificacion_prueba.py
Todo joya
Todo joya
Todo joya
Todo joya
Analisis completo de la base de datos de YTrain
Cantidad de imagenes analizadas:
21
Introduce numero de la foto: 1

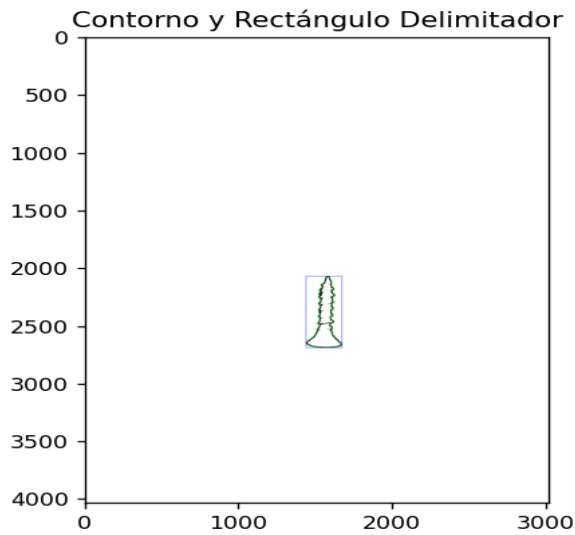
Inicializacion KNN
Prediccion para KNN con K=1:
Tornillo

Predicciones para KNN con K=5:
Tornillo
Tornillo
Tornillo
Tornillo
Tornillo

Inicializacion KMeans
KMeans Convergió en 2 iteraciones
Tornillo media: [0.0006823662359326753, 3.6497962247392325e-08, 6.834119022825598e-17, 12185713.0, 14108.0, 0.7693605391153143]
Tuerca media: [0.0006831794396023298, 3.659450005726013e-08, 6.040713582840045e-17, 12185713.0, 14108.0, 0.7693605391153143]
Arandela media: [0.0006880357924538366, 3.711966715989368e-08, 8.871554142281759e-16, 1917201.0, 5596.0, 0.7693467069880668]
Clavo media: [0.0006843784054994077, 3.660593660626732e-08, 3.868569878763888e-16, 12185713.0, 14108.0, 0.7693605391153143]

Prediccion para KMeans:
Tornillo
```

Luego como es un tornillo arroja la medida del mismo, aquí se muestra la imagen con el rectángulo delimitador que se usa para obtener la longitud del objeto:



```
Área del contorno: 176.5  
Rectángulo delimitador: x = 1437, y = 2069, w = 234, h = 620  
La medida es 20.064724919093848 mm  
PS D:\DB\IA_Proyecto> █
```

Lo que es bastante acertado:



2) Ejemplo clavo

El clavo analizado es el siguiente:



La clasificación arroja lo siguiente:

```
PS D:\DB\IA_Proyecto> & C:/Users/miled/AppData/Local/Programs/Python/Python311/python.exe d:/DB/IA_Proyecto/clasificacion_prueba.py
Todo joya
Todo joya
Todo joya
Todo joya
Analisis completo de la base de datos de YTrain
Cantidad de imagenes analizadas:
21
Introduce numero de la foto: 7

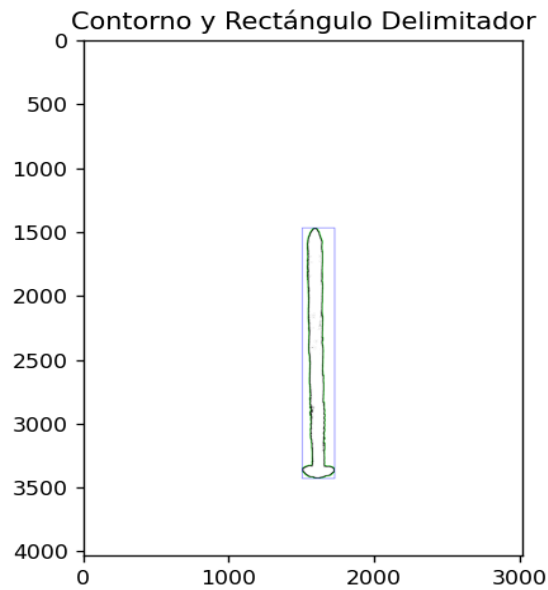
Inicializacion KNN
Prediccion para KNN con K=1:
Clavo

Predicciones para KNN con K=5:
Clavo
Clavo
Clavo
Clavo
Tuerca

Inicializacion KMeans
KMeans Convergió en 2 iteraciones
Tornillo media: [0.0006823662359326753, 3.6497962247392325e-08, 6.834119022825598e-17, 12185713.0, 14108.0, 0.7693605391153143]
Tuerca media: [0.0006831794396023298, 3.659450005726013e-08, 6.040713582840047e-17, 12185713.0, 14108.0, 0.7693605391153143]
Arandela media: [0.0006880357924538366, 3.711966715989368e-08, 8.871554142281759e-16, 1917201.0, 5596.0, 0.7693467069880668]
Clavo media: [0.0006843784054994077, 3.660593660626732e-08, 3.868569878763888e-16, 12185713.0, 14108.0, 0.7693605391153143]

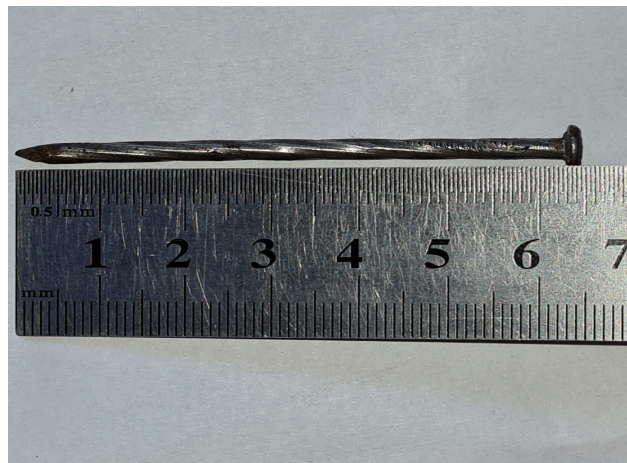
Prediccion para KMeans:
Clavo
```

Luego como es un tornillo arroja la medida del mismo, aquí se muestra la imagen con el rectángulo delimitador que se usa para obtener la longitud del objeto:



```
Área del contorno: 135.5  
Rectángulo delimitador: x = 1502, y = 1467, w = 223, h = 1962  
La medida es 63.495145631067956 mm  
PS D:\DB\IA_Proyecto>
```

Comparando con la medida real:



3) Ejemplo arandela

La arandela analizada es la siguiente:



Mediante la clasificación se obtiene:

```
PS D:\DB\IA_Proyecto> & C:/Users/miled/AppData/Local/Programs/Python/Python311/python.exe d:/DB/IA_Proyecto/clasificacion_prueba.py
Todo joya
Todo joya
Todo joya
Todo joya
Analisis completo de la base de datos de YTrain
Cantidad de imagenes analizadas:
21
Introduce numero de la foto: 3

Inicializacion KNN
Prediccion para KNN con K=1:
Arandela

Predicciones para KNN con K=5:
Arandela
Arandela
Arandela
Arandela
Arandela

Inicializacion KMeans
KMeans Convergíó en 2 iteraciones
Tornillo media: [0.0006823662359326754, 3.6497962247392325e-08, 6.834119022825598e-17, 12185713.0, 14108.0, 0.7693605391153143]
Tuerca media: [0.0006831794396023298, 3.659450005726013e-08, 6.040713582840047e-17, 12185713.0, 14108.0, 0.7693605391153143]
Arandela media: [0.0006880357924538366, 3.711966715989368e-08, 8.871554142281759e-16, 1917201.0, 5596.0, 0.7693467069880668]
Clavo media: [0.0006843784054994077, 3.660593660626732e-08, 3.868569878763887e-16, 12185713.0, 14108.0, 0.7693605391153143]

Prediccion para KMeans:
Arandela
PS D:\DB\IA_Proyecto>
```

4) Ejemplo tuerca

La tuerca analizada es la siguiente:



La clasificación nos brinda el siguiente resultado:

```
PS D:\DB\IA_Proyecto> & C:/Users/miled/AppData/Local/Programs/Python/Python311/python.exe d:/DB/IA_Proyecto/clasificacion_prueba.py
Todo joya
Todo joya
Todo joya
Todo joya
Analisis completo de la base de datos de YTrain
Cantidad de imagenes analizadas:
21
Introduce numero de la foto: 2

Inicializacion KNN
Prediccion para KNN con K=1:
Tuerca

Predicciones para KNN con K=5:
Tuerca
Tuerca
Tuerca
Tuerca
Tuerca

Inicializacion KMeans
KMeans Convergíó en 2 iteraciones
Tornillo media: [0.0006823662359326753, 3.6497962247392325e-08, 6.834119022825598e-17, 12185713.0, 14108.0, 0.7693605391153143]
Tuerca media: [0.0006831794396023298, 3.659450005726013e-08, 6.040713582840045e-17, 12185713.0, 14108.0, 0.7693605391153143]
Arandela media: [0.0006880357924538366, 3.711966715989368e-08, 8.871554142281759e-16, 1917201.0, 5596.0, 0.7693467069880668]
Clavo media: [0.0006843784054994077, 3.660593660626732e-08, 3.868569878763888e-16, 12185713.0, 14108.0, 0.7693605391153143]

Prediccion para KMeans:
Tuerca
```

Conclusiones

Se puede concluir que ambos métodos aplicados, KNN y K-Means, son bastante eficientes a la hora de clasificar los distintos elementos que se hallan en las fotografías. El funcionamiento de los mismos se ve muy influenciado por la forma en que se procesan las imágenes, así como la manera en que es tomada la foto. Las distintas horas del día, con sus respectivos cambios de luz pueden ser más o menos beneficiosos, también podríamos incluir en este dilema la posición y distancia de la cámara y color del fondo. Los mayores contratiempos los tuve debido a estos problemas antes mencionados.

La mayor parte de las veces que la predicción falla lo hace al tener que distinguir arandelas y tuercas, esto mejora un poco si se logra tomar una porción del espesor de esta última.

Algunas posibles mejoras son: conectar la cámara con la computadora mediante una dirección IP para evitar tener que mandar la imagen capturada por mensaje y luego descargarla; crear una interfaz que maneje todos los programas directamente para no tener que estar haciendo cambios en el código y por último podrían utilizarse redes neuronales para agilizar el trabajo y hasta tal vez hacer la clasificación en tiempo real. También podría utilizarse un método de medición más eficiente la relación de píxeles.

Bibliografía

- Inteligencia Artificial. Un Enfoque Moderno por Stuart Russell - Peter Norvig.
- <https://docs.opencv.org>
- <https://numpy.org/>
- <https://www.delftstack.com/es/howto/python/opencv-moments/>